

Luigi at the center of your pipelines

Resilience Without Complexity: WPipe vs. Luigi

The context

Did you know Luigi's biggest problem is its dependence on a central scheduler to keep state? If the scheduler goes down, your pipeline crumbles. 🧱 At WPipe, we took a different path: SQLite WAL Checkpoints.

Why engineers shift to wpipe

- Pipelines are Python: versionable, testable, reviewable in Git.
- State persisted to SQLite WAL at every step; resume, don't restart.
- <50MB RAM. No broker, no daemon, no metadata server.

Code-first

Pipelines are Python: versionable, testable, reviewable in Git.

Checkpoints

State persisted to SQLite WAL at every step; resume, don't restart.

Independently lean

<50MB RAM. No broker, no daemon, no metadata server.

What it looks like

```
from wpipe import Pipeline, step

@step(name="build", retry_count=2, retry_delay=5)
def build(data):
    target = compile_targets(data["source"])
    return {"artifacts": target}

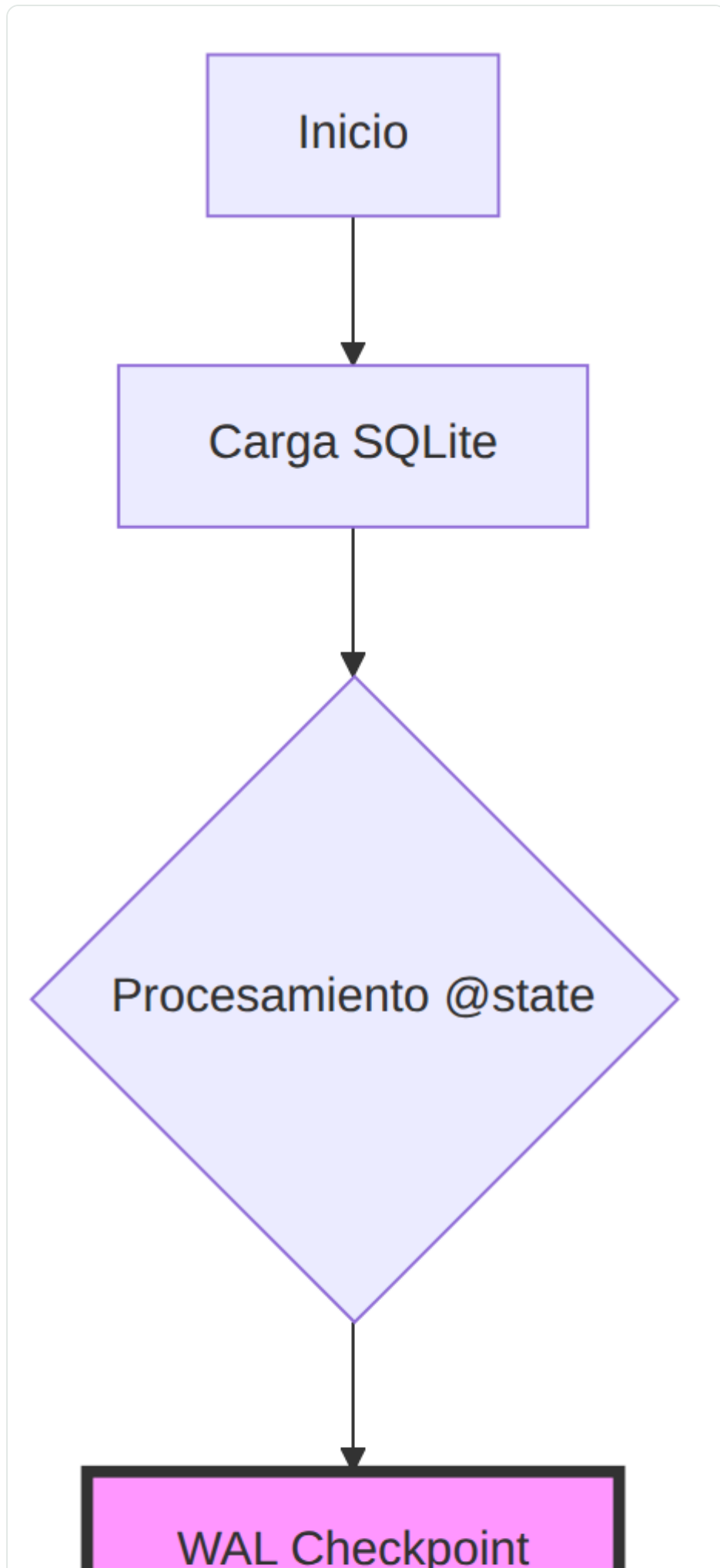
pipe = Pipeline(pipeline_name="luigi_like")
pipe.set_steps([build])
```

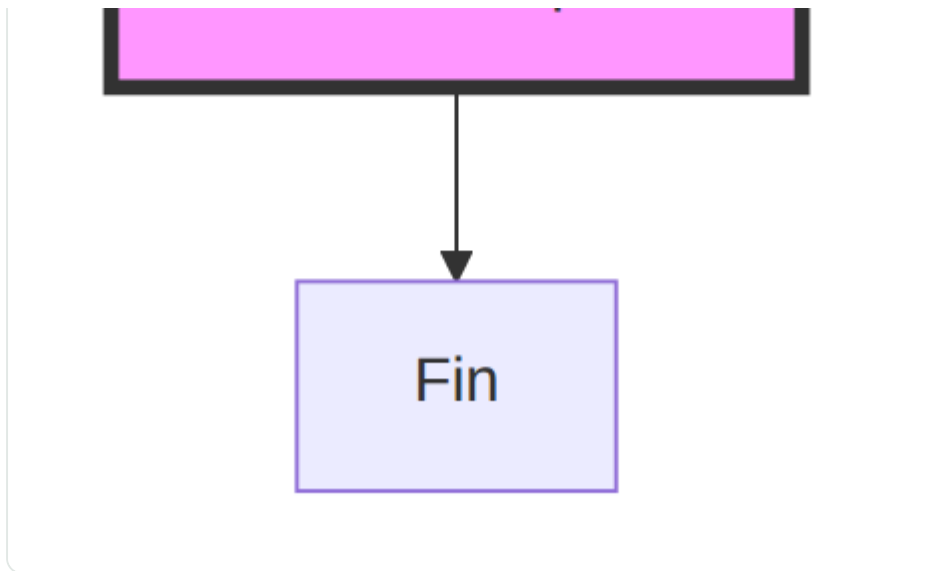
At a glance

Feature	WPipe	Luigi
Data Integrity	SQLite WAL (Atomic)	File-system / Central DB
Scalability	Zero-infra, Edge Ready	Centralized Heavy
RAM Usage	< 50MB (Consistent)	> 2GB (Spiky)
Auto-Docs	Built-in Mermaid	Luigi Visualizer

Under the hood

The flow behind the post, as wpipe renders it from code.





+117k downloads

A community of efficiency-first developers.

<50MB RAM

Fits containers, VMs, and Raspberry Pi-class hardware.

SQLite WAL

Industrial durability without a database to administer.

Next step

wpipe is a Python library — try it in your next pipeline:

```
pip install wpipe
```

Steps stay functions you already understand. Docs, diagrams, and dashboards follow. Explore Luigi workflows at wpipe.readthedocs.io.

wpipe · Python-native orchestration with SQLite WAL checkpoints, retries, timeouts, parallelism, and self-documenting flows. +117k downloads.